

aa0ec019feab3851d592248674b416

3. Proposal

3.1. Shard container registry separately to Cells architecture

Due to its extensive and in general highly scalable horizontal architecture it should be evaluated if the GitLab container registry should be run not in Cell, but in a Cluster and be scaled independently.

This might be easier, but would definitely not offer the same amount of data isolation.

3.2. Run container registry within a Cell

It appears that except `/jwt/auth` which would likely have to be processed by Router (to decode scope) the container registry could be run as a local service of a Cell.

The actual data at least in case of GitLab.com is not forwarded via registry, but rather served directly from Object Storage / CDN.

Its design encodes container repository image in a URL that is easily routable.

It appears that we could re-use the same stateless Router service in front of container registry to serve manifests and blobs redirect.

The only downside is increased complexity of managing standalone registry for each Cell, but this might be desired approach.

4. Evaluation

There do not seem to be any theoretical problems with running GitLab container registry in a Cell.

It seems that the service can be easily made routable to work well.

The practical complexities are around managing a complex service from an infrastructure side.

Since multiple Cells can be run on the same top-level domain, and the Docker client stores authentication tokens per-hostname, users may not be able to access resources from one Cell while logged in to another on the same hostname.

To authenticate with the container registry in a Cells environment, users will need to run:

```
shell
docker login gitlab.example.com
```

The username can be anything; it is not used by GitLab's container registry. The "password" must be one of:

- Personal access token
- Project access token
- Group access token

Per the current container registry authentication process.

Note that Deploy token is listed as an available password for the container registry, but since these tokens are not routeable, they will only work with the legacy cell. The user's GitLab username and password can also be used to authenticate with the container registry, but this authentication method also does not contain routing information, so will only work with the legacy cell. For the first iteration, only routing to the legacy cell and default organization will be supported for all token types.

The docker client will then submit this username and password combination to the `/jwt/auth` endpoint in GitLab Rails using HTTP Basic Auth.

The Cells http router will be able to determine the correct Cell to route this request to by supporting HTTP Basic Auth requests using access tokens as the password. Support for this is currently being built.

The returned JWT will contain enough information (something like `scope: cell-1`) for the Cells Registry Router to route the authenticated requests to the correct container registry / cell.

4.1. Pros

4.2. Cons

SECTION: engineering/architecture/design-documents/cells/impacted_features/contributions-forks.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

The forking workflow allows users to copy existing Project sources into their own namespace of choice (personal or Group).

1. Definition

The forking workflow is a common workflow with various usage patterns:

- It allows users to contribute back to an upstream Project.
- It persists repositories into their personal namespace.
- Users can copy a Project to make changes and release it as a modified Project.

Forks allow users not having write access to a parent Project to make changes.

The forking workflow is especially important for the open-source community to contribute back

to public Projects.

However, it is equally important in some companies that prefer a strong split of responsibilities and tighter access control.

The access to a Project is restricted to a designated list of developers.

Forks enable:

- Tighter control of who can modify the upstream Project.
- Split of responsibilities: Parent Project might use CI configuration connecting to production systems.
- To run CI pipelines in the context of a fork in a much more restrictive environment.
- To consider all forks to be unvetted which reduces risks of leaking secrets, or any other information tied to the Project.

The forking model is problematic in a Cells architecture for the following reasons:

- Forks are clones of existing repositories. Forks could be created across different Organizations, Cells and Gitaly shards.
- Users can create merge requests and contribute back to an upstream Project. This upstream Project might be in a different Organization and Cell.
- The merge request CI pipeline is executed in the context of the source Project, but presented in the context of the target Project.

2. Data exploration

From a data exploration, we retrieved the following information about existing forks:

- Roughly 1.8m forks exist on GitLab.com at the moment.
- The majority of forks are under a personal namespace (82%).
- We were expecting a minimal use of forks within the same top-level Group and/or organization. Forking is only necessary for users who don't have permissions to access a Project. Inside companies we wouldn't expect teams to use forking workflows much unless they for some reason have different permissions across different team members. The data showed that only 9% of fork relationships have matching ultimate parent namespace identifiers (top-level Groups and personal namespaces). The other 91% of fork relationships are forked across different top-level namespaces. When trying to match top-level Groups to an identifiable company, we saw that:
 - 3% of forked Projects are forked from an upstream Project in the same organization.
 - 83% of forked Projects do not have an identifiable organization related to either up or downstream Project.
 - The remaining 14% are forked from a source Project within a different company.
- 9% of top-level Groups (95k) with activity in the last 12 months have a project with a fork relationship, compared to 5% of top-level Groups (91k) with no activity in the last 12 months. We expect these top-level Groups to be impacted by Cells.

3. Proposals

3.1. Forks are created in a dedicated contribution space of the current Organization

Instead of creating Projects across Organizations, forks are created in a contribution space tied to the Organization.

A contribution space is similar to a personal namespace but rather than existing in the default Organization, it exists within the Organization someone is trying to contribute to.

Example:

- Any User that can view an Organization (all Users for public Organizations) can create a contribution space in the Organization. This is a dedicated namespace where they can create forks of Projects in that Organization. For example for Produce Inc. it could be `gitlab.com/organization/produce-inc/@ayufan`.
- To create a contribution space we do not require membership of an Organization as this would prevent open source workflows where contributors are able to fork and create a merge request without ever being invited to a Group or Project. We strictly respect visibility, so Users would not be able to create a fork in a private Organization without first being invited.
- When creating a fork for a Project Users will only be presented with the option to create forks in Groups that are part of the Organization. We will also give Users the option to create a contribution space and put the fork there. Today there is also a "Create a group" option when creating a fork. This functionality would also be limited to creating a new group in the organization to store the new fork.
- In order to support Users that want to fork without contributing back we might consider an option to create an unlinked fork in any namespace they have permission to write to.
- The User has as many contribution spaces as Organizations they contribute to.
- The User cannot create additional personal Projects within contribution spaces. Personal Projects can continue to be created in their personal namespace.
- The Organization can prevent or disable usage of contribution spaces. This would disable forking by anyone that does not belong to a Group within the Organization.
- All current forks are migrated into the contribution space of the User in an Organization. Because this may result in data loss when the fork also has links to data outside of the upstream Project we will also keep the personal Project around as archived and remove the fork relationship.
- All forks are part of the Organization.
- Forks are not federated features.
- The contribution space and forked Project do not share configuration with the parent Project.
- If the Organization is deleted, the Projects containing forks will be moved either to the default Organization or we'll create a new Organization to house them, which is essentially a ghost Organization of the former Organization.
- Data in contribution spaces do not contribute to customer usage from a billing perspective.
- Today we do not have organization-scoped runners but if we do implement that they will likely need special settings for how or if they can be used by contribution space projects.

3.2. Intra-cluster forks

This proposal implements forks as intra-cluster forks where communication is done via API between all trusted Cells of a cluster:

- Forks are created always in the context of a user's choice of Group.
- Forks are isolated from the Organization.
- Organization or Group owner could disable forking across Organizations, or forking in general.
- A merge request is created in the context of the target Project, referencing the external Project on another Cell.
- To target Project the merge reference is transferred that is used for presenting information

in context of the target Project.

- CI pipeline is fetched in the context of the source Project as it is today, the result is fetched into the merge request of the target Project.
- The Cell holding the target Project internally uses GraphQL to fetch the status of the source Project and includes in context of the information for merge request.

Pros:

- All existing forks continue to work as they are, as they are treated as intra-Cluster forks.

Cons:

- The purpose of Organizations is to provide strong isolation between Organizations. Allowing to fork across does break security boundaries.
- However, this is no different to the ability of users today to clone a repository to a local computer and push it to any repository of choice.
- Access control of the source Project can be lower than that of the target Project. Today, the system requires that in order to contribute back, the access level needs to be the same for fork and upstream Project.

3.3. Forks are created as internal Projects under current Projects

Instead of creating Projects across Organizations, forks are attachments to existing Projects. Each user forking a Project receives their unique Project.

Example:

- For Project: `gitlab.com/gitlab-org/gitlab`, forks would be created in `gitlab.com/gitlab-org/gitlab/@kamil-gitlab`.
- Forks are created in the context of the current Organization, they do not cross Organization boundaries and are managed by the Organization.
- Tied to the user (or any other user-provided name of the fork).
- Forks are not federated features.

Cons:

- Does not answer how to handle and migrate all existing forks.
- Might share current Group/Project settings, which could be breaking some security boundaries.

3.4. Forks are created in personal namespaces of the current Organization

Every User can potentially have a personal namespace in each public Organization. On the first visit to an Organization the User will receive a personal namespace scoped to that Organization.

A User can fork into a personal namespace provided the upstream repository is in the same Organization as the personal namespace.

Removal of an Organization will remove any personal namespaces in the Organization.

Pros:

- We re-use most existing code paths.

- We re-use most existing product design rules.
- Organization boundaries are naturally isolated.
- Multiple personal namespaces will mean Users can divide personal Projects across Organizations instead of having them mixed together.
- We expect most Users to work in one Organization, which means that the majority of them would not need to remember in which Organization they stored each of their personal Projects.

Cons:

- Redundant personal namespaces will be created. We expect to improve this in future iterations.
- Multiple personal namespaces could be difficult to navigate, especially when working across a large number of Organizations. We expect this to be an edge case.
- The life cycle of personal namespaces will be dependent on the Organization as is already the case for user accounts privately owned (such as Enterprise Users), and self-managed installations that are not public.
- Organization personal namespaces will need new URL paths.
- The legacy personal namespace path will need to be adapted.

URL path changes are under discussion.

4. Evaluation

We will follow 3.4. Forks are created in personal namespaces of the current Organization because it has already solved a lot of the hard problems.

The short falls of this solution like reworking URL paths or handling multiple personal namespaces are manageable and less critical than problems created through other alternative proposals.

5. Example

As an example, we will demonstrate the impact of this proposal for the case that we move gitlab-org/gitlab to a different Organization. gitlab-org/gitlab has over 8K forks.

Does this direction impact the canonical URLs of those forks?

Yes canonical URLs will change for forks.

Specific path changes are under discussion.

Existing Users that have forks in legacy personal namespaces and want to continue contributing merge requests, will be required to migrate their fork to their personal namespace in the source project Organization.

For example, a personal namespace fork at <https://gitlab.com/DylanGriffith/gitlab> will need to be migrated to <https://gitlab.com/-/organizations/gitlab-inc/@DylanGriffith/gitlab>.

We may offer automated ways to move this, but manually the process would involve:

1. Create the contribution space fork
1. Push your local branch from your original fork to the new fork
1. Recreate any merge request that was still open and you wanted to merge

Does it impact the Git URL of the repositories themselves?

Yes.

Specific path changes are under discussion.

Would any user action be required to accept their fork to be moved?

No. A fork can be moved within an Organization or to a contribution space.

If the Organization is public, then a user will have a personal namespace.

Can we make promises that we will not break the existing forks of public Projects hosted on GitLab.com?

Existing fork projects will not be deleted but their fork relationship will be removed when the source project is moved to another Organization.

The owner of the open source project will be made aware that they will disconnect their forks when they move the project which will require them to close all existing merge requests from those forks.

There will need to be some process for keeping the history from these merge requests while effectively losing the ability to collaborate on them or merge them.

In the case of gitlab-org/gitlab we will attempt to give as much notice of this process and make this process as transparent as possible.

When we make the decision to move this project to an Organization we will seek additional feedback about what would be the minimum amount of automated migrations necessary to be acceptable here.

But the workflow for contributors will change after the move so this will be a punctuated event regardless.

SECTION: engineering/architecture/design-documents/cells/impacted_features/data-migration.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement.

This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

It is essential for a Cells architecture to provide a way to migrate data out of big Cells into smaller ones.

This document describes various approaches to provide this type of split.

We also need to handle cases where data is already violating the expected isolation constraints of Cells, for example references cannot span multiple Organizations.

We know that existing features like linked issues allowed users to link issues across any

Projects regardless of their hierarchy.

There are many similar features.

All of this data will need to be migrated in some way before it can be split across different Cells.

This may mean some data needs to be deleted, or the feature needs to be changed and modelled slightly differently before we can properly split or migrate Organizations between Cells.

Having schema deviations across different Cells, which is a necessary consequence of different databases, will also impact our ability to migrate data between Cells.

Different schemas impact our ability to reliably replicate data across Cells and especially impact our ability to validate that the data is correctly replicated.

It might force us to only be able to move data between Cells when the schemas are all in sync (slowing down deployments and the rebalancing process) or possibly only migrate from newer to older schemas which would be complex.

1. Definition

2. Data flow

3. Proposal

3.1. Split large Cells

A single Cell can only be divided into many Cells.

This is based on the principle that it is easier to create an exact clone of an existing Cell in many replicas out of which some will be made authoritative once migrated.

Keeping those replicas up-to-date with Cell 0 is also much easier due to pre-existing replication solutions that can replicate the whole systems: Geo, PostgreSQL physical replication, etc.

1. All data of an Organization needs to not be divided across many Cells.

1. Split should be doable online.

1. New Cells cannot contain pre-existing data.

1. N Cells contain exact replica of Cell 0.

1. The data of Cell 0 is live replicated to as many Cells it needs to be split.

1. Once consensus is achieved between Cell 0 and N-Cells, the Organizations to be migrated away are marked as read-only cluster-wide.

1. The routes is updated on for all Organizations to be split to indicate an authoritative Cell holding the most recent data, like gitlab-org on cell-100.

1. The data for gitlab-org on Cell 0, and on other non-authoritative N-Cells are dormant and will be removed in the future.

1. All accesses to gitlab-org on a given Cell are validated about cellid of routes to ensure that given Cell is authoritative to handle the data.

More challenges of this proposal

1. There is no streaming replication capability for Elasticsearch, but you could snapshot the whole Elasticsearch index and recreate, but this takes hours.

It could be handled by pausing Elasticsearch indexing on the initial Cell during the migration as indexing downtime is not a big issue, but this still needs

to be coordinated with the migration process.

1. Syncing Redis, Gitaly, CI Postgres, Main Postgres, registry Postgres, other new data stores snapshots in an online system would likely lead to gaps without a long downtime. You need to choose a sync point and at the sync point you need to stop writes to perform the migration. The more data stores there are to migrate at the same time the longer the write downtime for the failover. We would also need to find a reliable place in the application to actually block updates to all these systems with a high degree of confidence. In the past we've only been confident by shutting down all Rails services because any Rails process could write directly to any of these at any time due to async workloads or other surprising code paths.
1. How to efficiently delete all the orphaned data. Locating all cibuilds associated with half the Organizations would be very expensive if we have to do joins. We haven't yet determined if we'd want to store an organizationid column on every table, but this is the kind of thing it would be helpful for.

3.2. Migrate Organization from an existing Cell

This is different to split, as we intend to perform logical and selective replication of data belonging to a single Organization.

Today this type of selective replication is only implemented by Gitaly where we can migrate Git repository from a single Gitaly node to another with minimal downtime.

In this model we would require identifying all resources belonging to a given Organization: database rows, object storage files, Git repositories, etc. and selectively copy them over to another (likely) existing Cell importing data into it.

Ideally ensuring that we can perform logical replication live of all changed data, but change similarly to split which Cell is authoritative for this Organization.

1. It is hard to identify all resources belonging to an Organization.
1. It requires either downtime for the Organization or a robust system to identify live changes made.
1. It likely will require a full database structure analysis (more robust than Project import/export) to perform selective PostgreSQL logical replication.

More challenges of this proposal

1. Logical replication is still not performant enough to keep up with our scale. Even if we could use logical replication we still don't have an efficient way to filter data related to a single Organization without joining all the way to the organizations table which will slow down logical replication dramatically.

4. Evaluation

4.1. Pros

4.2. Cons

SECTION: engineering/architecture/design-documents/cells/impacted_features/data-pipeline-ingestion.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

The Cells architecture will have a significant impact on the current data pipeline which exports data from Postgres to Snowflake for the use of data analytics. This data pipeline fulfils many use cases (i.e. SAAS Service ping, Gainsight metrics and Reporting and Analytics of the SAAS Platform).

1. Definition

2. Data flow

The current data pipeline is limited by not having the possibility to get data via a CDC mechanism (which leads to data quality issues) and works by polling the Postgres database and looking for new and updated records or fully extracting data for certain tables which causes a lot of overhead.

At the moment the data pipeline runs against two instances that get created from a snapshot of both the main and ci databases.

This is done to avoid workload on the production databases.

In the Cells architecture there will be more Postgres instances because of which the current pipeline couldn't scale to pull data from all the Postgres instances. Requirements around the data pipeline moving forward are as follows:

- We need a process that allows capturing all the CDC (insert, update and delete) from all Cells, scaling automatically with N number of Cells.
- We need to have (direct or indirect) access to database instances which allows it to do data catch up in case of major failure or root cause analysis for data anomalies.
- We need monitoring in place to alert any incident that can delay the data ingestion.

3. Proposal

4. Evaluation

4.1. Pros

4.2. Cons

SECTION: engineering/architecture/design-documents/cells/impacted_features/dependency-

proxy.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

GitLab Dependency Proxy is a feature allowing storage of Docker container images in GitLab for use by CI jobs and end users. It acts as a pull-through cache of upstream registries such as Docker Hub, enabling faster image builds and avoiding restricted rate limits by upstream services.

As Dependency Proxy is effectively a private image registry, it follows the same deployment pattern and challenges as the GitLab container registry.

1. Definition

The dependency proxy follows the same data usage and storage challenges as the container registry.

2. Data flow

The data flow for docker login is documented in the container registry project. The equivalent requests using curl are documented below.

2.1. Authorization request that is send by docker login

shell

curl \

--user "username:password" \

"https://gitlab/jwt/auth?clientid=docker&offline_token=true&service=containerregistry&scope=repository:gitlab-org/gitlab-build-images:push,pull"

Result is encoded and signed JWT token. Second base64 encoded string (split by .) contains JSON with authorized scopes.

json

```
{"auth_type": "none", "access": [{"type": "repository", "name": "gitlab-org/gitlab-build-images", "actions": ["pull"]}], "jti": "61ca2459-091c-4496-a3cf-01bac51d4dc8", "aud": "containerregistry", "iss": "omnibus-gitlab-issuer", "iat": 1669309469, "nbf": 166}
```

2.2. Docker client fetching tags

```
shell
curl \
-H "Accept: application/vnd.docker.distribution.manifest.v2+json" \
-H "Authorization: Bearer token" \
https://registry.gitlab.com/v2/gitlab-org/gitlab-build-images/tags/list

curl \
-H "Accept: application/vnd.docker.distribution.manifest.v2+json" \
-H "Authorization: Bearer token" \
https://registry.gitlab.com/v2/gitlab-org/gitlab-build-images/manifests/danger-ruby-2.6.6
```

2.3. Docker client fetching blobs and manifests

```
shell
curl \
-H "Accept: application/vnd.docker.distribution.manifest.v2+json" \
-H "Authorization: Bearer token" \
https://registry.gitlab.com/v2/gitlab-org/gitlab-build-
images/blobs/sha256:a3f2e1afa377d20897e08a85cae089393daa0ec019feab3851d592248674b416
```

3. Proposal

Dependency Proxy can be run within a cell, similar to the proposal for the container registry.

4. Evaluation

Since multiple Cells can be run on the same top-level domain, and the Docker client stores authentication tokens per-hostname, users may not be able to access resources from one Cell while logged in to another on the same hostname.

To authenticate with the dependency proxy in a Cells environment, users will need to run:

```
shell
docker login gitlab.example.com
```

Currently, using your GitLab username and password is supported by this flow. In a Cells architecture, this will only work for the legacy cell and default organization, since the username / password combination is not routeable.

The username can be anything; it is not used by dependency proxy. The "password" must be a Personal Access Token from a user, service account, or CI job token. The docker client will then submit this username and password combination to the /jwt/auth endpoint in GitLab Rails using HTTP Basic Auth.

The Cells http router will be able to determine the correct Cell to route this request to by supporting HTTP Basic Auth requests using access tokens as the password. Support for this is currently being built.

The returned JWT will contain enough information (something like scope: cell-1) for the Cells Registry Router to route the authenticated requests to the correct container registry / cell.

4.1. Pros

4.2. Cons

SECTION: engineering/architecture/design-documents/cells/impacted_features/explore.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement.

This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

Explore may not play a critical role in GitLab as it functions today, but GitLab today is not isolated. It is the isolation that makes Explore or some viable replacement necessary.

The existing Group and Project Explore will initially be scoped to an Organization. However, there is a need for a global Explore that spans across Organizations to support the discoverability of public Groups and Projects, in particular in the context of discovering open source Projects. See user feedback [here](#) and [here](#).

1. Definition

The Explore functionality helps users in discovering Groups and Projects. Unauthenticated Users are only able to explore public Groups and Projects, authenticated Users can see all the Groups and Projects that they have access to, including private and internal Groups and Projects.

2. Data flow

3. Proposal

The Explore feature problem falls under the broader umbrella of solving inter-Cell communication. This topic warrants deeper research.

Below are possible directions for further investigation.

3.1. Read only table mirror

- Create a sharedprojects table in the shared cluster-wide database.
- The model for this table is read-only. No inserts/updates/deletes are allowed.
- The table is filled with data (or a subset of data) from the Projects Cell-local table.
 - The write model Project (which is Cell-local) writes to the local database. We will primarily use this model for anything Cell-local.

- This data is synchronized with sharedprojects via a background job any time something changes.
- The data in sharedprojects is stored normalized, so that all the information necessary to display the Project Explore is there.
- The Project Explore (as of today) is part of an instance-wide functionality, since it's not namespaced to any organizations/groups.
- This section will read data using the read model for sharedprojects.
- Once the user clicks on a Project, they are redirected to the Cell containing the Organization.

Downsides:

- Need to have an explicit pattern to access instance-wide data. This however may be useful for admin functionalities too.
- The Project Explore may not be as rich in features as it is today (various filtering options, role you have on that Project, etc.).
- Extra complexity in managing CQRS.

3.2 Explore scoped to an Organization

The Project Explore and Group Explore are scoped to an Organization.

Downsides:

- No global discoverability of Groups and Projects.

4. Evaluation

The existing Group and Project Explore will initially be scoped to an Organization. Considering the current usage of the Explore feature, we deem this acceptable. Since all existing Users, Groups and Projects will initially be part of the default Organization, Groups and Projects will remain explorable and accessible as they are today. Only once existing Groups and Projects are moved out of the default Organization into different Organizations will this become a noticeable problem. Solutions to mitigate this are discussed in issue 418228. Ultimately, Explore could be replaced with a better search experience altogether.

4.1. Pros

- Initially the lack of discoverability will not be a problem.
- Only around 1.5% of all existing Users are using the Explore functionality on a monthly basis.

4.2. Cons

- The GitLab owned top-level Groups would be some of the first to be moved into their own Organization and thus be detached from the explorability of the default Organization.

SECTION: engineering/architecture/design-documents/cells/impacted_features/git-access.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

This document describes impact of Cells architecture on all Git access (over HTTPS and SSH) patterns providing explanation of how potentially those features should be changed to work well with Cells.

1. Definition

Git access is done throughout the application.

It can be an operation performed by the system (read Git repository) or by a user (create a new file via Web IDE, git clone or git push via command line).

The Cells architecture defines that all Git repositories will be local to the Cell, so no repository could be shared with another Cell.

The Cells architecture will require that any Git operation can only be handled by a Cell holding the data.

It means that any operation either via Web interface, API, or GraphQL needs to be routed to the correct Cell.

It means that any git clone or git push operation can only be performed in the context of a Cell.

2. Data flow

There are various operations performed today by GitLab on a Git repository.

This describes the data flow how they behave today to better represent the impact.

It appears that Git access does require changes only to a few endpoints that are scoped to a Project.

There appear to be different types of repositories:

- Project: assigned to Group
- Wiki: additional repository assigned to Project
- Design: similar to Wiki, additional repository assigned to Project
- Snippet: creates a virtual Project to hold repository, likely tied to the User

2.1. Git clone over HTTPS

Execution of: git clone over HTTPS

mermaid

sequenceDiagram

User ->> Workhorse: GET /gitlab-org/gitlab.git/info/refs?service=git-upload-pack

Workhorse ->> Rails: GET /gitlab-org/gitlab.git/info/refs?service=git-upload-pack

Rails ->> Workhorse: 200 OK
Workhorse ->> Gitaly: RPC InfoRefsUploadPack
Gitaly ->> User: Response
User ->> Workhorse: POST /gitlab-org/gitlab.git/git-upload-pack
Workhorse ->> Gitaly: RPC PostUploadPackWithSidechannel
Gitaly ->> User: Response

2.2. Git clone over SSH

Execution of: git clone over SSH

mermaid

sequenceDiagram

User ->> Git SSHD: ssh git@gitlab.com
Git SSHD ->> Rails: GET /api/v4/internal/authorizedkeys
Rails ->> Git SSHD: 200 OK (list of accepted SSH keys)
Git SSHD ->> User: Accept SSH
User ->> Git SSHD: git clone over SSH
Git SSHD ->> Rails: POST /api/v4/internal/allowed?project=/gitlab-org/gitlab.git&service=git-upload-pack
Rails ->> Git SSHD: 200 OK
Git SSHD ->> Gitaly: RPC SSHUploadPackWithSidechannel
Gitaly ->> User: Response

2.3. Git push over HTTPS

Execution of: git push over HTTPS

mermaid

sequenceDiagram

User ->> Workhorse: GET /gitlab-org/gitlab.git/info/refs?service=git-receive-pack
Workhorse ->> Rails: GET /gitlab-org/gitlab.git/info/refs?service=git-receive-pack
Rails ->> Workhorse: 200 OK
Workhorse ->> Gitaly: RPC PostReceivePack
Gitaly ->> Rails: POST /api/v4/internal/allowed?glrepository=project-111&service=git-receive-pack
Gitaly ->> Rails: POST /api/v4/internal/prereceive?glrepository=project-111
Gitaly ->> Rails: POST /api/v4/internal/postreceive?glrepository=project-111
Gitaly ->> User: Response

2.4. Git push over SSHD

Execution of: git clone over SSH

mermaid

sequenceDiagram

User ->> Git SSHD: ssh git@gitlab.com


```

Git SSHD ->> Rails: GET /api/v4/internal/authorizedkeys
Rails ->> Git SSHD: 200 OK (list of accepted SSH keys)
Git SSHD ->> User: Accept SSH
User ->> Git SSHD: git clone over SSH
Git SSHD ->> Rails: POST /api/v4/internal/allowed?project=/gitlab-
org/gitlab.git&service=git-receive-pack
Rails ->> Git SSHD: 200 OK
Git SSHD ->> Gitaly: RPC ReceivePack
Gitaly ->> Rails: POST /api/v4/internal/allowed?glrepository=project-111
Gitaly ->> Rails: POST /api/v4/internal/prereceive?glrepository=project-111
Gitaly ->> Rails: POST /api/v4/internal/postreceive?glrepository=project-111
Gitaly ->> User: Response

```

2.5. Create commit via Web

Execution of Add CHANGELOG to repository:

```

mermaid
sequenceDiagram
    Web ->> Puma: POST /gitlab-org/gitlab/-/create/main
    Puma ->> Gitaly: RPC TreeEntry
    Gitaly ->> Rails: POST /api/v4/internal/allowed?glrepository=project-111
    Gitaly ->> Rails: POST /api/v4/internal/prereceive?glrepository=project-111
    Gitaly ->> Rails: POST /api/v4/internal/postreceive?glrepository=project-111
    Gitaly ->> Puma: Response
    Puma ->> Web: See CHANGELOG

```

3. Proposal

The Cells stateless router proposal requires that any ambiguous path (that is not routable) will be made routable.

It means that at least the following paths will have to be updated to introduce a routable entity (Project, Group, or Organization).

Change:

```

- /api/v4/internal/allowed => /api/v4/internal/projects/<glrepository>/allowed
- /api/v4/internal/prereceive => /api/v4/internal/projects/<glrepository>/prereceive
- /api/v4/internal/postreceive => /api/v4/internal/projects/<glrepository>/postreceive
- /api/v4/internal/lfsauthenticate => /api/v4/internal/projects/<glrepository>/lfsauthenticate

```

Where:

```

- glrepository can be project-1111 (Gitlab::GIRepository)
- glrepository in some cases might be a full path to repository as executed by GitLab Shell
(/gitlab-org/gitlab.git)

```

4. Evaluation

Supporting Git repositories if a Cell can access only its own repositories does not appear to be complex.

The one major complication is supporting snippets, but this likely falls in the same category as for the approach to support a user's Personal Namespace.

4.1. Pros

1. The API used for supporting HTTPS/SSH and Hooks are well defined and can easily be made routable.

4.2. Cons

1. The sharing of repositories objects is limited to the given Cell and Gitaly node.
1. Cross-Cells forks are likely impossible to be supported (discover: How this works today across different Gitaly node).

5. Forking and object pools

One of the biggest struggles that needs to be addressed with the Cells architecture is how to handle forking. At present, Gitaly utilizes object pools to provide deduplication of fork storage. If forks are not created on the same storage node as the upstream repository that is being forked, we end up with significant storage inefficiencies as we will effectively have two complete copies of the repository and we will not be able to utilize object pools to improve performance.

The storage nodes from one Cell cannot talk to the storage nodes of another Cell, making forking across Cells impossible. Therefore, it will be necessary to ensure that forked repositories end up in the same Cell (and on the same Gitaly nodes) as their upstream parent repository. This will also enable Gitaly to continue to utilize object pools to provide storage and performance efficiency.

5.1. How this works today

Single Gitaly storage node

Currently, for a GitLab instance backed with a single Gitaly storage node, forking works just fine.

Any forks must reside on the same storage node as there is only one, and therefore object deduplication (and object pools) all function as expected.

Sharded Gitaly storage

A sharded Gitaly storage is when multiple Gitaly storage nodes are attached to a single instance, and repositories are assigned based on a priority weighting between the nodes.

Since Gitaly knows how to do cross-storage fetches, forking across shards works without issue.

Gitaly Cluster

For Gitaly cluster, we recently resolved the issue of object pools not being created on the

same storage nodes as the parent repository. This enables forking to work correctly from an efficiency perspective (can share an object pool) and from an object deduplication perspective (Git can properly deduplicate storage).

SECTION: engineering/architecture/design-documents/cells/impacted_features/gitlab-pages.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

TL;DR

1. Definition
2. Data flow
3. Proposal
4. Evaluation
- 4.1. Pros
- 4.2. Cons

SECTION: engineering/architecture/design-documents/cells/impacted_features/global-search.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

When we introduce multiple Cells we intend to isolate all services related to those Cells. This will include Elasticsearch which means our current global search functionality will not work.

It may be possible to implement aggregated search across all Cells, but it is unlikely to be performant to do fan-out searches across all Cells especially once you start to do pagination which requires setting the correct offset and page number for each search.

1. Definition
2. Data flow
3. Proposal

Likely the first versions of Cells will not support global searches.
Later, we may consider if building global searches to support popular use cases is worthwhile.

4. Evaluation
- 4.1. Pros
- 4.2. Cons

SECTION: engineering/architecture/design-documents/cells/impacted_features/graphql.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

GitLab extensively uses GraphQL to perform efficient data query operations. GraphQL due to its nature is not directly routable. The way GitLab uses it calls the `/api/graphql` endpoint, and only the query or mutation of the body request might define where the data can be accessed.

1. Definition
2. Data flow
3. Proposal

There are at least two main ways to implement GraphQL in a Cells architecture.

- 3.1. GraphQL routable by endpoint

Change `/api/graphql` to `/api/organization/<organization>/graphql`.

- This breaks all existing usages of `/api/graphql` endpoint because the API URI is changed.

- 3.2. GraphQL routable by body

As part of router parse GraphQL body to find a routable entity, like project.

- This still makes the GraphQL query be executed only in context of a given Cell and not allowing the data to be merged.

json

Good example

```
{
  project(fullPath:"gitlab-org/gitlab") {
    id
    description
  }
}
```

Bad example, since Merge Request is not routable

```
{
  mergeRequest(id: 1111) {
    iid
    description
  }
}
```

3.3. Merging GraphQL Proxy

Implement as part of router GraphQL Proxy which can parse body and merge results from many Cells.

- This might make pagination hard to achieve, or we might assume that we execute many queries of which results are merged across all Cells.

json

```
{
  project(fullPath:"gitlab-org/gitlab"){
    id, description
  }
  group(fullPath:"gitlab-com") {
    id, description
  }
}
```

4. Evaluation

4.1. Pros

4.2. Cons

SECTION: engineering/architecture/design-documents/cells/impacted_features/group-transfer.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

TL;DR

1. Definition
2. Data flow
3. Proposal

There is an investigation to solve this problem using direct transfer.

4. Evaluation

4.1. Pros

4.2. Cons

Direct transfer does not migrate users and users cannot exist on more than one Cell. This means in Cells 1.0, for migrations across Cells, any user contributions will be assigned to the user performing the import.

SECTION: engineering/architecture/design-documents/cells/impacted_features/issues.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

TL;DR

1. Definition
2. Data flow
3. Proposal

4. Evaluation

4.1. Pros

4.2. Cons

SECTION: engineering/architecture/design-documents/cells/impacted_features/merge-requests.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

TL;DR

1. Definition

2. Data flow

3. Proposal

4. Evaluation

4.1. Pros

4.2. Cons

SECTION: engineering/architecture/design-documents/cells/impacted_features/organizations.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

One of the major designs of a Cells architecture is strong isolation between Groups. Organizations as described by the Organization blueprint provides a way to have plausible UX for joining together many Groups that are isolated from the rest of the system.

1. Definition

Cells do require that all Groups and Projects of a single Organization can only be stored on a single Cell because a Cell can only access data that it holds locally and has very limited capabilities to read information from other Cells.

Cells with Organizations do require strong isolation between Organizations.

It will have significant implications on various user-facing features, like Todos, dropdowns allowing to select Projects, references to other issues or Projects, or any other social functions present at GitLab.

Today those functions were able to reference anything in the whole system.

With the introduction of Organizations this will be forbidden.

This problem definition aims to answer effort and implications required to add strong isolation between Organizations to the system, including features affected and their data processing flow.

The purpose is to ensure that our solution when implemented consistently avoids data leakage between Organizations residing on a single Cell.

2. Proposal

See the Organization blueprint.

SECTION: engineering/architecture/design-documents/cells/impacted_features/personal-access-tokens.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement.

This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

1. Definition

Personal Access Tokens (PATs) associated with a User are a way for Users to interact with the API of GitLab to perform operations.

PATs today are scoped to the User, and can access all Groups that a User has access to.

2. Data flow

3. Proposal

Various approaches have been discussed in the following issues / merge requests:

- 428717
- 428542
- !136939

3.1. Organization-scoped PATs

Pros:

- Can be managed entirely from Rails application.
- Increased security. PAT is limited only to Organization.
- API requests for a particular Cell can be routed based on the PAT.

Cons:

- Different PAT needed for different Organizations.
- Automations involving multiple projects across different Organizations will require multiple PATs to work - one for each Organization.
- Cannot tell at a glance if PAT will apply to a certain Project/Namespace.

3.2. Routing

The Topology Service will claim the tokens to facilitate routing requests to the owning Cell. However, this functionality is only expected to be available in Cells 1.5.

To avoid imposing additional requirements on the Topology Service team for Cells 1.0, a token prefix will be used for routing.

The prefix will be associated with the owning Cell, allowing routing to function via the Topology Service without claiming individual tokens.

However, this approach will increase the workload for teams working on gitlab-rails and SREs.

The prefix needs to be added to the following tokens:

- Personal access Token (including its derivatives: impersonation Token, project access token, group access token)
- OAuth access and refresh token
- RSS / calendar feed tokens
- Incoming email tokens
- Runner token
- CI Job token
- Pipeline trigger tokens
- Deploy token

4. Alternative approaches considered

4.1. Cluster-wide PATs

Pros:

- User does not have to worry about which scope the PAT applies to.

Cons:

- User has to worry about wide-ranging scope of PAT (e.g. separation of personal items versus work items).
- Organization cannot limit scope of PAT to only their Organization.
- Increases complexity. All cluster-wide data likely will be moved to a separate data access layer.

4.2. Cluster-wide PATs with optional Organization scoping

This is a combination of option 3.1 and 4.1 where the PATs are created cluster-wide but can be optionally scoped to an Organization.

For initial development, the implication would be to support clusterwide PATs (including addressing any scalability challenges) and eventually adding organization as a token scope (similar to API or read/write scopes).

Pros:

- Flexible and secure giving users (or admins) control over the PAT scope

Cons:

- Risk around discovering scalability challenges when accessing, sharing or updating clusterwide data.

5. Evaluation

5.1. Pros

5.2. Cons

SECTION: engineering/architecture/design-documents/cells/impacted_features/personal-namespaces.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement.

This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

Personal Namespaces do not easily fit with our overall architecture in Cells, because the Cells architecture depends on all data belonging to a single Organization.

When Users are allowed to work across multiple Organizations there is no natural fit for picking a single Organization to store personal Namespaces and their Projects.

One important engineering constraint in Cells will be that data belonging to one Organization should not be linked to data belonging to another Organization. Specifically, functionality in GitLab should be scoped to a single Organization at a time. This presents a challenge for personal Namespaces as forking is one of the important workloads for personal Namespaces. Functionality related to forking and the UI that presents forked MRs to users will often require data from both the downstream and upstream Projects at the same time. Implementing such functionality would be very difficult if that data belonged to different Organizations stored on different Cells. This is especially the case with the merge request, as it is one of the most complicated and performance critical features in GitLab.

Today personal Namespaces serve two purposes that are mostly non-overlapping:

1. They provide a place for users to create personal Projects that aren't expected to receive contributions from other people. This use case saves them from having to create a Group just for themselves.
1. They provide a default place for a user to put any forks they create when contributing to Projects where they don't have permission to push a branch. This again saves them from needing to create a Group just to store these forks. But the primary user need here is because they can't push branches to the upstream Project so they create a fork and contribute merge requests from the fork.

1. Definition

A personal Namespace is based on a username and provided when a user creates an account. Users can create personal Projects under their personal Namespace.

2. Data flow

3. Proposal

As described above, personal Namespaces serve two purposes today:

1. A place for users to store their Projects to save them from creating a Group.
1. A place for users to store forks when they want to contribute to a Project where they don't have permission to push a branch.

In this proposal we will only focus on (1) and assume that (2) will be replaced by suitable workflows described in Cells: Contributions: Forks.

Since we plan to move away from using personal Namespaces as a home for storing forks, we can assume that the main remaining use case does not need to support cross-Organization linking.

3.1. One personal Namespace that can move between Organizations

For existing Users personal Namespaces will exist within the default Organization in the short term.

This implies that all Users will, at first, have an association to the default Organization via their personal Namespace.

When a new Organization is created, new Users can be created in that Organization as well. A new User's personal Namespace will be associated with that new Organization, rather than the default.

Also, Users can become members of Organizations other than the default Organization.

In this case, they will have to switch to the default Organization to access their personal Namespace until we have defined a way for them to move their personal Namespace into a different Home Organization.

Doing so may necessitate that personal Namespaces are converted to Groups before being moved.

When an Organization is deleted, we will need to decide what should happen with the personal Namespaces associated with it.

If we go this route, there may be breakage similar to what will happen to when we move Groups or Projects into their own Organization, though the full impact may need further investigation.

This decision, however, means that existing personal Namespaces that were used as forks to contribute to some upstream Project will become disconnected from the upstream as soon as the upstream moves into an Organization.

On GitLab.com 10% of all projects in personal Namespaces are forks.

This may be a slightly disruptive workflow but as long as the forks are mainly just storing branches used in merge requests then it may be reasonable to ask the affected users to recreate the fork in the context of the Organization.

We will further explore the idea of a contribution space to give Users a place to store forks when they want to contribute to a Project where they don't have permission to push a branch. That discussion will be handled as part of the larger discussion of the Cells impact on forks.

Pros:

- Easy access to personal Namespace via a User's Home Organization. We expect most Users to work in only a single Organization.
- Contribution graph would remain intact for Users that only work in one Organization, because their personal and organizational activity would be aggregated as part of the same Organization.

Cons:

- A transfer mechanism to move personal Namespaces between Organizations would need to be built, which is extremely complex. This would be in violation of the current Cells architecture, because Organizations can be located on different Cells. To make this possible, we would need to break Organization isolation.
- High risk that transfer between Organizations would lead to breaking connections and data loss.
- Converting personal Namespaces to Groups before transfer is not a straightforward process.

3.2. One personal Namespace that remains in the default Organization

For existing Users personal Namespaces will exist within the default Organization in the short term.

This implies that all Users will, at first, have an association to the default Organization via their personal Namespace.

New Users joining GitLab as part of an Organization other than the default Organization would

also receive a personal Namespace in the default Organization.
Organization other than the default Organization would not contain personal Namespaces.

Pros:

- No transfer mechanism necessary.

Cons:

- Users that are part of multiple Organizations need to remember that their personal content is stored in the default Organization. To access it, they would have to switch back to the default Organization.
- New Users might not understand why they are part of the default Organization.
- Some impact on the User Profile page. No personal Projects would be shown in Organizations other than the default Organization. This would result in a lot of whitespace on the page. The Personal projects list would need to be reworked as well.

3.3. One personal Namespace in each Organization

For existing Users personal Namespaces will exist within the default Organization in the short term.

As new Organizations are created, Users receive additional personal Namespaces for each Organization they interact with.

For instance, when a User views a Group or Project in an Organization, a personal Namespace is created.

This is necessary to ensure that community contributors will be able to continue contributing to Organizations without becoming a member.

Pros:

- Content of personal Projects is owned by the Organization. Low risk for enterprises to leak content outside of their organizational boundaries.
- No transfer mechanism necessary.
- No changes to the User Profile page are necessary.
- Users can keep personal Projects in each Organization they work in.
- No contribution space for forking necessary.
- No need to make the default Organization function differently than other Organizations.

Cons:

- Users have to remember which personal content they store in each Organization.
- Personal content would be owned by the Organization. However, this would be similar to how self-managed operates today and might be desired by enterprises.

3.4. Discontinue personal Namespaces

All existing personal Namespaces are converted into Groups.

The Group path is identical to the current username.

Upon Organization release, these Groups would be part of the default Organization.

We disconnect Users from the requirement of having personal Namespaces, making the User a truly

global entity.

Pros:

- Users would receive the ability to organize personal Projects into Groups, which is a highly requested feature.
- No need to create personal Namespaces upon User creation.
- No path changes necessary for existing personal Projects.

Cons:

- A concept of personal Groups would need to be established.
- It is unclear how @-mentions would work. Currently it is possible to tag individual Users and Groups. Following the existing logic all group members belonging to a personal Group would be tagged.
- Significant impact on the User Profile page. Personal Projects would be disconnected from the User Profile page and possibly replaced by new functionality to highlight specific Projects selected by the User (via starring or pinning).
- It is unclear whether Groups could be migrated between Organizations using the same mechanism as needed to migrate top-level Groups. We expect this functionality to be highly limited at least in the mid-term. Similar transfer limitations as described in section 3.1. are expected.

4. Evaluation

We will begin by making the personal namespace optional for Organizations. The goal of this iteration is to disable personal namespaces for any Organization other than the default Organization, so that customers who do not want to use personal namespaces can already move to Organizations. The first phase will only change the Ruby on Rails model relationships in preparation for further changes at the user-facing level.

We need to split the concept of a User Profile and a personal namespace now that a User is cluster-wide and a User's personal namespace must be Cell-local. It is likely we will discontinue personal namespaces in favor of Groups.

SECTION: engineering/architecture/design-documents/cells/impacted_features/project-transfer.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

TL;DR

1. Definition
2. Data flow
3. Proposal
4. Evaluation
- 4.1. Pros
- 4.2. Cons

SECTION: engineering/architecture/design-documents/cells/impacted_features/router-endpoints-classification.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement.

This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

Classification of all endpoints is essential to properly route requests hitting the load balancer of a GitLab installation to a Cell that can serve it.
Each Cell should be able to decode each request and classify which Cell it belongs to.

GitLab currently implements hundreds of endpoints.

This document tries to describe various techniques that can be implemented to allow the Rails to provide this information efficiently.

1. Definition
2. Data flow
3. Proposal
4. Evaluation
- 4.1. Pros
- 4.2. Cons

SECTION: engineering/architecture/design-documents/cells/impacted_features/schema-changes.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement.

This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

When we introduce multiple Cells that own their own databa